Iteration 3: Optimizations Document

Optimization 1: Database Read Optimization (Polling to Adaptive Fetch)

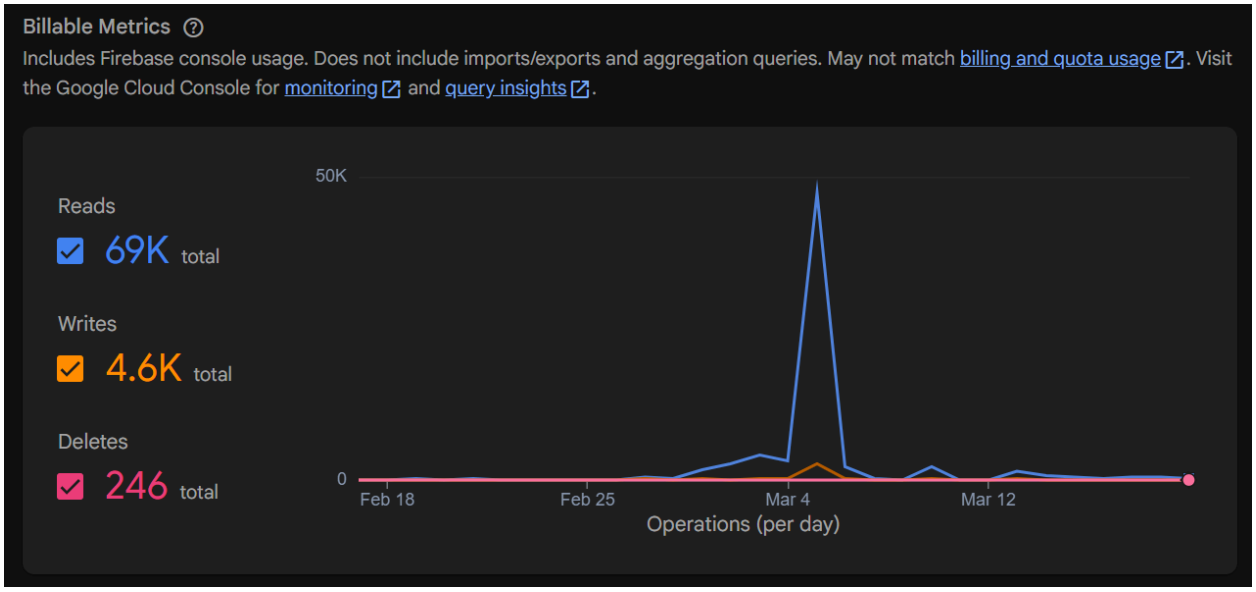
The Problem & Code Smell

Identified Issue: Inefficient Data Retrieval and High Resource Consumption.

During Iteration 2, our real-time chat implementation relied on continuous, unconditional polling. The client requested the last 21 messages from Firestore every few seconds. This created a severe performance bottleneck and resource drain, resulting in over 21,443 document reads in a single day (consuming roughly 42% of our daily Firebase quota), even when no new messages were sent.

What the Design Looked Like Before

- **Client Behavior:** The frontend continuously polled the `/chat2/*/messages` endpoint at a fixed interval.
- **Backend Behavior:** Every poll triggered a full database read of the targeted message documents.
- **Result:** Massive redundancy. The system repeatedly fetched unchanged, heavy message payloads, scaling aggressively and unsustainably as more users kept chat tabs open.



Iteration 3: Optimizations Document

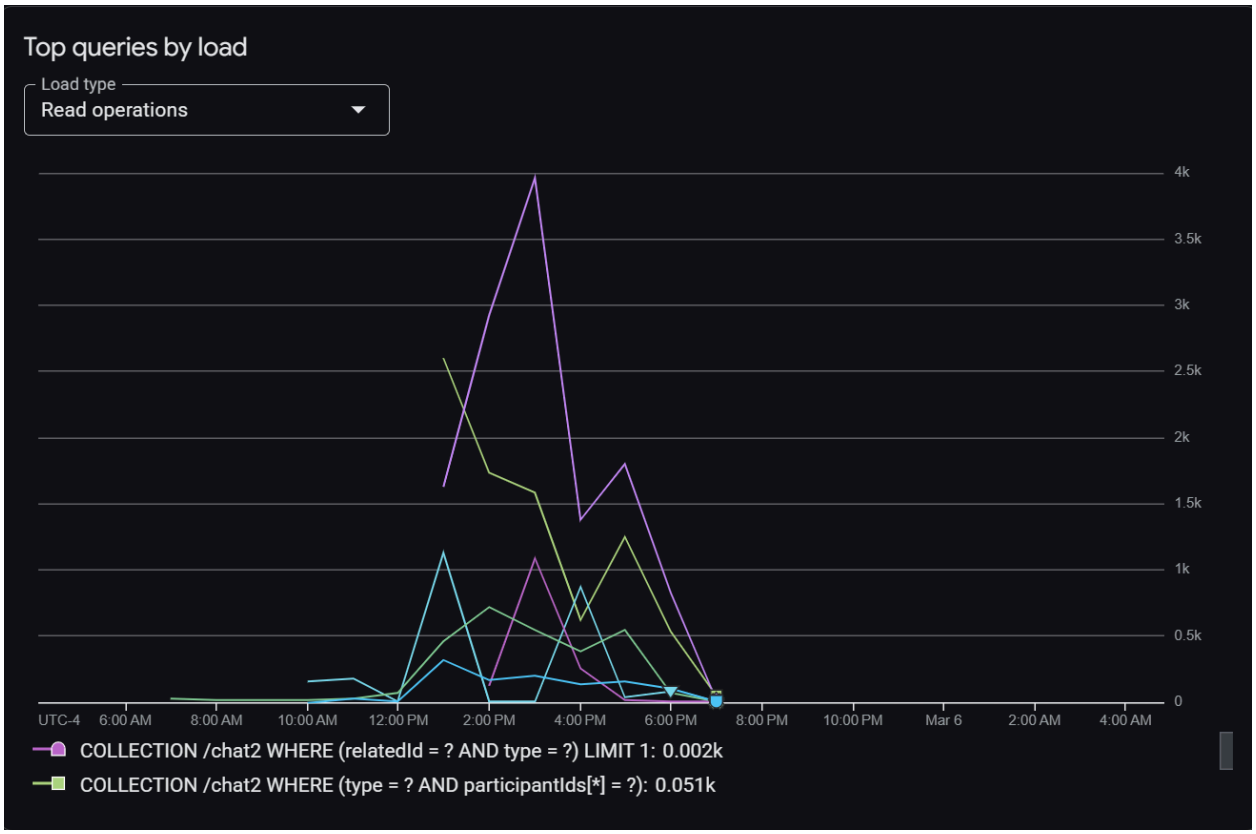


Image showing the specific chat endpoints/services with the high READ Requests to DB.

The Optimization Applied

We replaced the "Continuous Full Polling" architecture with a **"Two-Step Adaptive Fetch" (Probe-then-Fetch)** strategy.

1. **Denormalization:** We updated the Chat document schema to track minimal state, adding `latestMessageTimestamp` and `latestMessageId`.
2. **Lightweight Metadata Endpoint:** We created a new, ultra-cheap endpoint that only returns the chat's metadata (timestamp and ID) rather than message contents.
3. **Client-Side State Tracking:** The client now maintains local state (`lastTimestampEpochMillis`). It probes the metadata endpoint using an adaptive polling interval (e.g., 3-5 seconds if active, 30-60 seconds if backgrounded) with a ±10% jitter to prevent backend load spikes.

How it Looks Now

Instead of pulling full data blindly, the client now asks the server, "Has anything changed since my last timestamp?" via a single, cheap document read.

- **If NO change:** The client skips the full fetch entirely.
- **If YES (timestamp changed):** The client makes a secondary call to fetch the actual message payload.

Iteration 3: Optimizations Document

This optimization drastically reduced our Firestore read footprint while maintaining the illusion of real-time responsiveness for the user.

Image showing much lower read/writes to DB after the optimization.

Optimization 2: Authentication State Hardening & Centralization

The Problem & Code Smell

Identified Issue: Scattered Logic, Incomplete Teardown, and Stale Session Vulnerabilities.

In Iteration 2, the authentication logic was fragmented across different components. The system suffered from "stale state," where logging out or timing out did not completely securely tear down the user's session data across the browser. Furthermore, route guards were superficial.

What the Design Looked Like Before

- **Route Guarding:** Protected routes only checked if a `STUDY_BUDDY_TOKEN_KEY` string *existed* in `localStorage`. It did not verify if the token was actually valid or expired.
- **Incomplete Teardown:** The logout function only removed the main token key. It left behind the `DEV_ACTOR_STORAGE_KEY` and allowed chat sessions to persist.
- **Inactivity Timeout:** The inactivity timer successfully redirected users to a timeout screen, but it failed to clear the authentication state, meaning a user could potentially navigate back to a protected page without signing back in.

Iteration 3: Optimizations Document

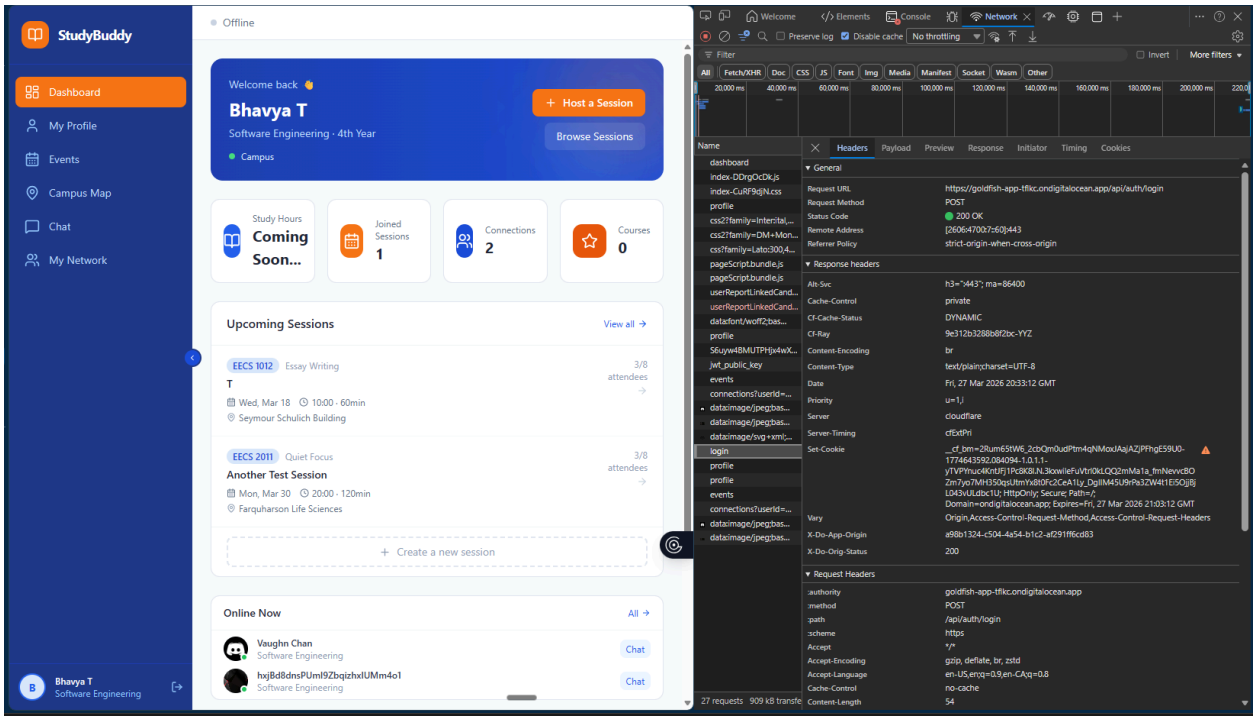


Image shows the “login” token which user receives upon sign-in

The Optimization Applied

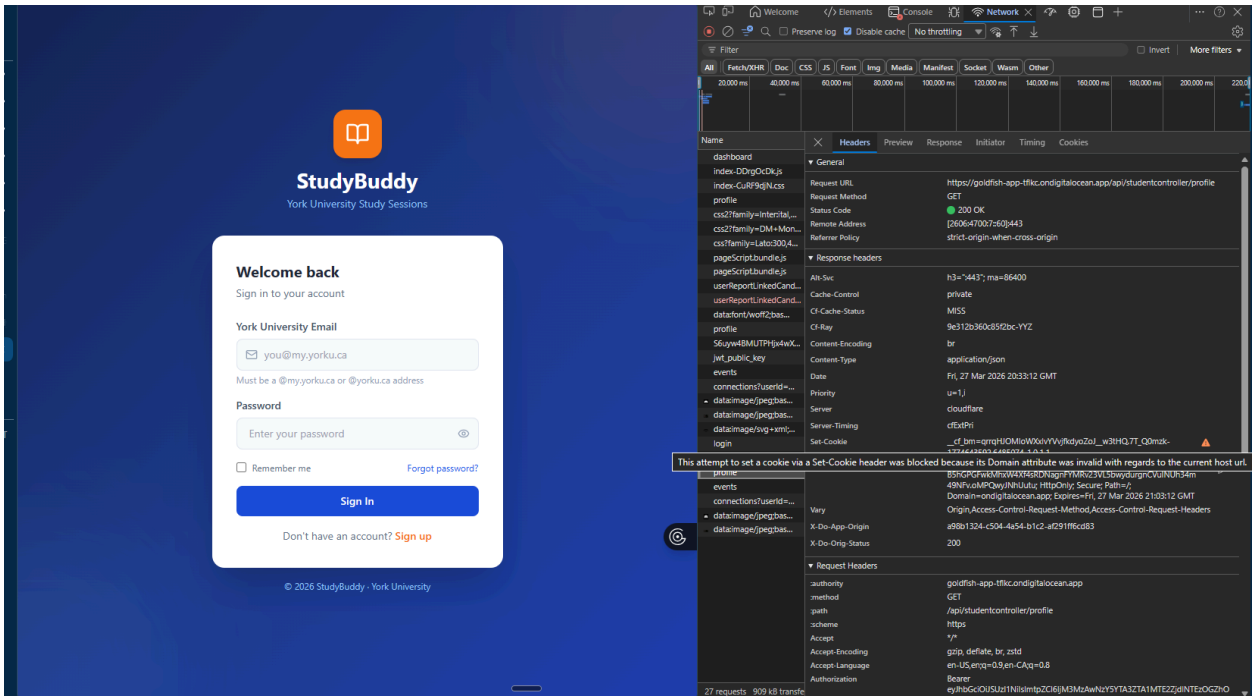
We applied a **Centralized State Management** optimization to our authentication flow.

1. **Centralized Auth Utility:** We abstracted all auth-related local and session storage interactions into a single, unified `auth.ts` service (featuring `setAuthToken`, `getAuthToken`, and `clearAuthState`).
2. **Hardened Route Guards:** We upgraded the protected-route logic to actively validate the token against the backend `/profile` endpoint rather than just checking for local existence.
3. **Unified Teardown:** We optimized `Login.tsx`, `Layout.tsx` (Logout button), and `useInactivityTimer.ts` to strictly route through `clearAuthState()`, executing a complete sweep of all session and local storage data.

How it Looks Now

Authentication state is now tightly controlled and predictable. If a user clicks "Sign Out" or triggers the inactivity timeout, `clearAuthState()` completely wipes all access tokens and actor IDs. If a user attempts to manually navigate to a protected route (like `/chat`) with a stale or invalid token, the hardened route guard validates it against the backend, detects the invalidity, automatically clears the local state, and forces a redirect to the Sign-In page.

Iteration 3: Optimizations Document



The orange warning message shows that the cookie was invalid after sign out, preventing unauthorized login access.